

UNIVERSIDADE FEDERAL DO PARANÁ  
Trabalho de Arquitetura de Computadores (CI1212)  
Análise de Memória Cache

**Autores**

Davi Alves Sampaio GRR20255653

Thalísia Arianna Fernandes Fleck GRR20256088

Haico de Toledo Boehs GRR20253482

Melissa Goulart Kemp GRR20255413

Analuiza Alves da Cruz GRR20254471

Curitiba

2026

# INTRODUÇÃO

A memória cache é uma peça importantíssima para os processadores modernos, pois reduz o tempo de acesso aos dados e melhora o desempenho geral das aplicações. Dessa forma, compreender seu funcionamento é essencial para analisar otimizações de software e hardware.

À vista disso, esse trabalho pretende analisar o desempenho da cache, estimando o tamanho das linhas e dos níveis de cache, além de mensurar o grau em que o *cache miss* pode prejudicar o tempo de execução das tarefas.

Para isso, fizemos três testes: o primeiro para estimar o tamanho de uma linha de cache, o segundo para determinar a latência - ou seja, tempo para completar uma instrução desde o começo até o fim - e com isso estimar o tamanho das memórias cache (L1, L2 e L3) e da RAM, e, por fim, o terceiro teste para analisar a eficiência da multiplicação de matrizes em blocos.

Os dois primeiros testes foram realizados individualmente nas máquinas de três participantes do grupo, de forma a garantir maior consistência na análise dos resultados em diferentes arquiteturas de processadores. Essa abordagem permite observar comportamentos consistentes entre sistemas com variações nos níveis de cache e latência de memória. Além disso, diferenças de arquitetura entre os computadores podem influenciar o tamanho e a organização das caches, afetando diretamente o impacto de otimizações baseadas em blocos, o que reforça a necessidade de realizar testes em múltiplos ambientes.

## CACHE E DADOS

### O QUE É CACHE?

A cache é uma partição da hierarquia de memória que serve como mediadora entre as informações (dados, instruções etc.) da memória RAM e o processador que precisa processá-las. Essa memória armazena dados de níveis mais baixos da hierarquia para facilitar os acessos e permitir que o sistema continue em atividade enquanto espera novos dados chegarem, em vez de ficar ocioso enquanto aguarda dados provenientes da RAM ou do HD/SSD. Os componentes físicos da cache são diferentes daqueles de camadas mais inferiores: a RAM, por exemplo, costuma ser feita de DRAM, uma tecnologia mais lenta e de maior densidade, porém mais barata; por outro lado, geralmente a cache é feita de SRAM, que é mais rápida, porém menos densa e, por isso, mais custosa. A cache funciona como um buffer para a DRAM e transporta as informações mais acessadas entre os níveis da hierarquia de memória.

### COMO A CACHE FUNCIONA?

O processo de captura de dados em cache funciona da seguinte maneira:

- a) O dispositivo solicita um dado;
- b) Primeiro é checado se o dado necessário já está na cache do nível superior ou não, sendo um **cache hit** se sim e um **cache miss** caso contrário (neste último caso, é preciso pegar o dado da parte mais inferior e lenta);
- c) Uma vez que o dado solicitado é obtido da fonte primária, caso não esteja presente na cache, ele é armazenado nela para facilitar acessos futuros;
- d) Em caso de cache cheia, é usada alguma política de substituição, geralmente a *Least Recently Used* (LRU), em que o dado que foi solicitado e ainda não está na cache é inserido no lugar do dado que não é usado há mais tempo.

## Dados das máquinas usadas para o Benchmark:

### TAMANHO DA LINHA DA CACHE:

Todas as máquinas testadas obtiveram 64 bytes como tamanho da linha cache, tamanho comum para os processadores de hoje em dia.

### TAMANHO DA HIERARQUIA CACHE:

comando utilizado: lscpu | grep cache

#### PC1 (Thalisia), CPU: Ryzen 5 3600, S.O\*: Linux Mint

|           |                        |
|-----------|------------------------|
| L1d cache | 192 KiB (6 *Instances) |
| L1i cache | 192 KiB (6 Instances)  |
| L2 cache  | 3 MiB (6 Instances)    |
| L3 cache  | 32 MiB (2 Instances)   |

#### PC2 (Davi), CPU: Ryzen 7 5700 U, S.O: Arch Linux

|           |                       |
|-----------|-----------------------|
| L1d cache | 256 KiB (8 Instances) |
| L1i cache | 256 KiB (8 Instances) |
| L2 cache  | 4 MiB (8 Instances)   |
| L3 cache  | 8 MiB (2 Instances)   |

#### PC3 (Melissa), CPU: Intel Core I9-10850K, S.O: Windows

|           |                        |
|-----------|------------------------|
| L1d cache | 320 KiB (10 Instances) |
| L1i cache | 320 KiB (10 Instances) |
| L2 cache  | 2.5 MiB (10 Instances) |
| L3 cache  | 20 MiB (1 Instance)    |

*\*S.O: Sistema Operacional*

*\*Instances: As caches L1 e L2 são privadas por núcleo, ou seja, são instanciadas individualmente para cada core do processador. Já a cache L3 pode ser compartilhada ou fisicamente segmentada, dependendo da arquitetura da CPU.*

## TESTE 1

### Método:

Neste primeiro teste, implementamos em C um programa para analisar o impacto do tamanho do salto entre acessos consecutivos à memória sobre a latência de acesso. Temos os seguintes arquivos: o cabeçalho **cache\_line.h**, sua implementação **cache\_line.c**, uma main.c para teste, o makefile e os arquivos dados.csv e **graph.py** para plotar o gráfico.

A implementação utiliza um array fixo de 8 MB - grande para garantir que seja maior do que as caches menores do processador - e tem uma função única para testar a linha. O teste é feito em strides, que são saltos que variam na execução.

Os bytes são convertidos em elementos do array (ex.: 16 bytes = salto de 4 em 4), e a proporção de cache hits e cache misses depende do tamanho do salto. Por exemplo, quando o salto é menor do que a linha de cache, há maior localidade espacial e, conseqüentemente, menos cache misses.

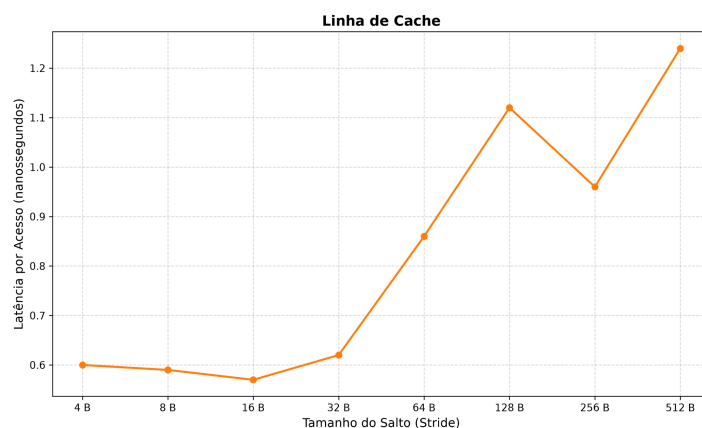
Já quando o salto é maior ou igual ao tamanho da linha da cache, a taxa de cache miss aumenta significativamente, pois cada acesso tende a exigir uma busca em níveis inferiores da hierarquia de memória. À medida que o stride aumenta além do tamanho da linha de cache, a localidade espacial é perdida, fazendo com que quase todos os acessos resultem em cache miss. Como consequência, a latência tende a se estabilizar em um patamar elevado, reduzindo a variação observada no gráfico. A latência tende a aumentar com o salto (variação : é a quantidade de bytes do stride).

“**dados.csv**” é um arquivo de dados que guarda a latência para cada stride, ou seja, para cada salto de n bytes. Ambos são diretamente proporcionais.

“**graph.py**”: plota um gráfico que traça o tamanho da linha de cache visualmente. Esse arquivo lê o “**dados.csv**”, que contém uma coluna de salto\_bytes (tamanho do pulo) e outra de latencia\_ns (tempo de acesso para cada tamanho de salto) e cria uma escala em log na base 2. Deixar a base logarítmica em 2 ajuda a uniformizar o espaçamento entre os valores e impedir que haja um desnível entre os valores menores e os maiores. Por fim, o gráfico é salvo em formato png.

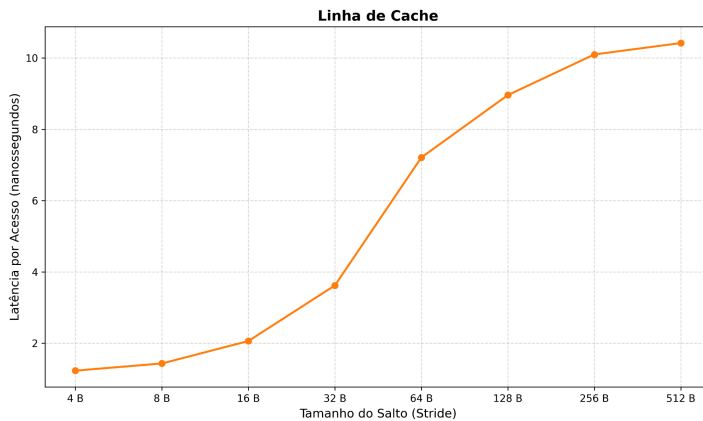
### Resultados:

PC1 (Thalisia), CPU: Ryzen 5 3600, S.O: Linux Mint



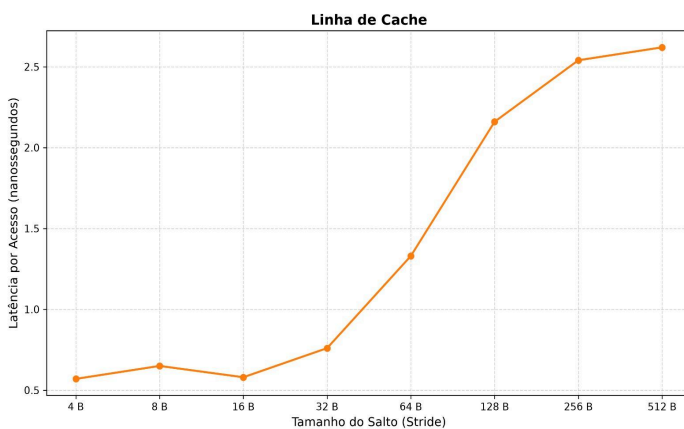
|            |       |                |
|------------|-------|----------------|
| Salto: 4   | Bytes | Tempo: 0.60 ns |
| Salto: 8   | Bytes | Tempo: 0.59 ns |
| Salto: 16  | Bytes | Tempo: 0.57 ns |
| Salto: 32  | Bytes | Tempo: 0.62 ns |
| Salto: 64  | Bytes | Tempo: 0.86 ns |
| Salto: 128 | Bytes | Tempo: 1.12 ns |
| Salto: 256 | Bytes | Tempo: 0.96 ns |
| Salto: 512 | Bytes | Tempo: 1.24 ns |

## PC2 (Davi), CPU: Ryzen 7 5700 U, S.O: Arch Linux



|            |       |                 |
|------------|-------|-----------------|
| Salto: 4   | Bytes | Tempo: 1.23 ns  |
| Salto: 8   | Bytes | Tempo: 1.43 ns  |
| Salto: 16  | Bytes | Tempo: 2.06 ns  |
| Salto: 32  | Bytes | Tempo: 3.62 ns  |
| Salto: 64  | Bytes | Tempo: 7.21 ns  |
| Salto: 128 | Bytes | Tempo: 8.96 ns  |
| Salto: 256 | Bytes | Tempo: 10.10 ns |
| Salto: 512 | Bytes | Tempo: 10.43 ns |

## PC3 (Melissa), CPU: Intel Core i9-10850K, S.O: Windows



|            |       |                |
|------------|-------|----------------|
| Salto: 4   | Bytes | Tempo: 0.57 ns |
| Salto: 8   | Bytes | Tempo: 0.65 ns |
| Salto: 16  | Bytes | Tempo: 0.58 ns |
| Salto: 32  | Bytes | Tempo: 0.76 ns |
| Salto: 64  | Bytes | Tempo: 1.33 ns |
| Salto: 128 | Bytes | Tempo: 2.16 ns |
| Salto: 256 | Bytes | Tempo: 2.54 ns |
| Salto: 512 | Bytes | Tempo: 2.62 ns |

## ANÁLISE DOS RESULTADOS

Como mostrado nas especificações dos processadores, o PC1 tem a maior capacidade para o nível mais inferior (L3 de 32 MiB), porém o menor tamanho para o nível mais superior (L1 de 192 KiB). Já o PC2 tem a maior capacidade para o nível mediano (L2 de 4 MiB), mas o pior nível inferior (L3 de 8 MiB apenas). Por fim, o PC3 tem o maior nível mais superior (L1 de 312 KiB) e o pior nível mediano (L2 de 2.5 MiB). Isso influencia diretamente no tempo levado em cada etapa para cada processador: o PC3 é o melhor de todos em latência para os saltos iniciais - até 32 bytes de stride -, e a partir dessa mesma quantidade o PC1 começa a sobressair em rapidez, justamente por ter o nível L3 de maior capacidade.

## TESTE 2

### Método

O código foi desenvolvido com base na latência média de acesso a arrays de diferentes tamanhos, a fim de estimar o limite de cada cache da CPU e quão importantes são os *cache misses* no tempo de execução final. Fizemos três arquivos: o **cache\_latency.c**, o **graph.py** e o **dados.csv**.

**cache\_latency.c**: é um arquivo que mede a latência de acesso à memória para calcular o tamanho das caches L1, L2, L3 e da memória principal (RAM), a partir de um loop que testa tamanhos de array que crescem gradualmente.

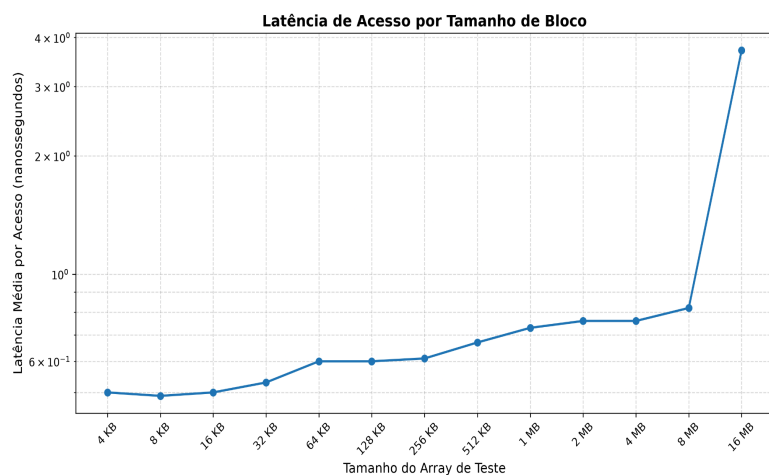
Também é feito um truque para avaliar o impacto da *cache miss* no tempo de execução: fazemos com que a CPU acesse o array de forma não sequencial, utilizando um salto fixo de 16 bytes (stride), de modo que primeiro acessa array[0], depois array[16], array[32], e assim por diante. Dessa forma, quando acaba o tamanho dessa array do bloco que já está disponível na cache, o programa precisa buscar uma nova linha que não está na cache, o que força o processador a trabalhar no pior cenário possível com várias *cache misses*. Aos poucos os níveis mais superiores, como o L1, são estourados e o programa precisa ir para L2, L3 até o último nível inferior. A ideia desse programa de teste é ver em que ponto essas mudanças de desempenho ocorrem.

“graph.py”: faz a leitura do dados.csv, que contém testes de latência, e calcula a média de latência para cada tamanho de bloco, e daí plota um gráfico com escalas logarítmicas para indicar as transições entre os níveis L1, L2, L3 e a própria memória RAM. O gráfico gerado é salvo em png.

“dados.csv”: é o código usado para que o código em Python gere um gráfico sem linhas. É um arquivo de dados.

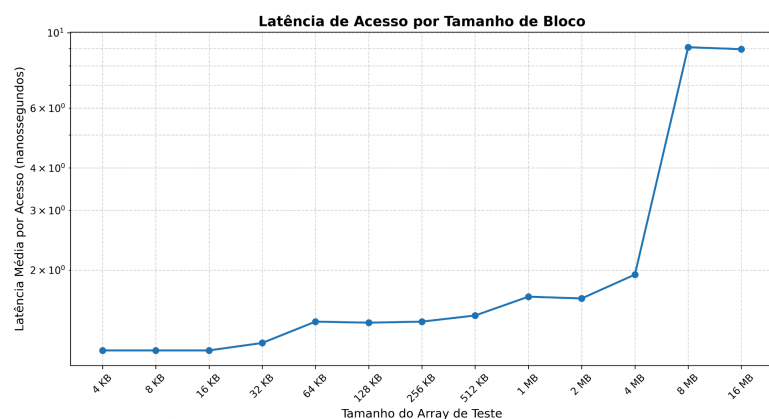
## Resultados (1 rodada)

PC1 (Thalisia), CPU: Ryzen 5 3600, S.O: Linux Mint



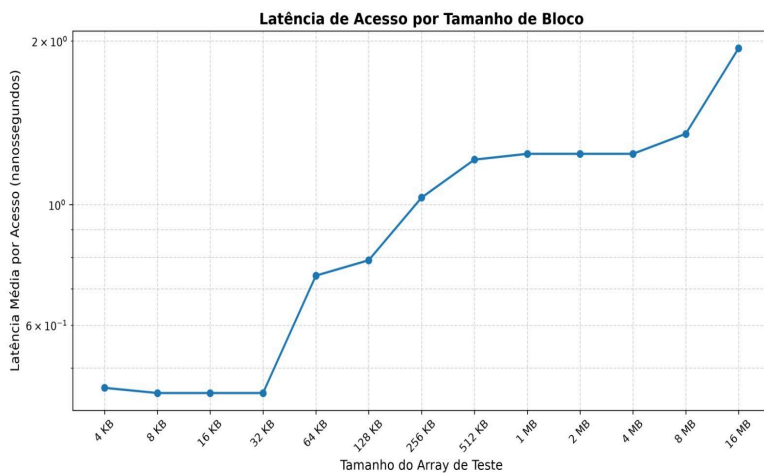
|              |    |                         |         |
|--------------|----|-------------------------|---------|
| Array: 4     | KB | Tempo médio por acesso: | 0.50 ns |
| Array: 8     | KB | Tempo médio por acesso: | 0.49 ns |
| Array: 16    | KB | Tempo médio por acesso: | 0.50 ns |
| Array: 32    | KB | Tempo médio por acesso: | 0.53 ns |
| Array: 64    | KB | Tempo médio por acesso: | 0.60 ns |
| Array: 128   | KB | Tempo médio por acesso: | 0.60 ns |
| Array: 256   | KB | Tempo médio por acesso: | 0.61 ns |
| Array: 512   | KB | Tempo médio por acesso: | 0.67 ns |
| Array: 1024  | KB | Tempo médio por acesso: | 0.73 ns |
| Array: 2048  | KB | Tempo médio por acesso: | 0.76 ns |
| Array: 4096  | KB | Tempo médio por acesso: | 0.76 ns |
| Array: 8192  | KB | Tempo médio por acesso: | 0.82 ns |
| Array: 16384 | KB | Tempo médio por acesso: | 3.71 ns |

PC2 (Davi), CPU: Ryzen 7 5700 U, S.O: Arch Linux



|              |    |                         |         |
|--------------|----|-------------------------|---------|
| Array: 4     | KB | Tempo médio por acesso: | 1.16 ns |
| Array: 8     | KB | Tempo médio por acesso: | 1.16 ns |
| Array: 16    | KB | Tempo médio por acesso: | 1.16 ns |
| Array: 32    | KB | Tempo médio por acesso: | 1.22 ns |
| Array: 64    | KB | Tempo médio por acesso: | 1.41 ns |
| Array: 128   | KB | Tempo médio por acesso: | 1.40 ns |
| Array: 256   | KB | Tempo médio por acesso: | 1.41 ns |
| Array: 512   | KB | Tempo médio por acesso: | 1.47 ns |
| Array: 1024  | KB | Tempo médio por acesso: | 1.67 ns |
| Array: 2048  | KB | Tempo médio por acesso: | 1.65 ns |
| Array: 4096  | KB | Tempo médio por acesso: | 1.94 ns |
| Array: 8192  | KB | Tempo médio por acesso: | 9.07 ns |
| Array: 16384 | KB | Tempo médio por acesso: | 8.97 ns |

PC3 (Melissa), CPU: Intel Core I9-10850K, S.O: Windows



|              |    |                         |         |
|--------------|----|-------------------------|---------|
| Array: 4     | KB | Tempo médio por acesso: | 0.46 ns |
| Array: 8     | KB | Tempo médio por acesso: | 0.45 ns |
| Array: 16    | KB | Tempo médio por acesso: | 0.45 ns |
| Array: 32    | KB | Tempo médio por acesso: | 0.45 ns |
| Array: 64    | KB | Tempo médio por acesso: | 0.74 ns |
| Array: 128   | KB | Tempo médio por acesso: | 0.79 ns |
| Array: 256   | KB | Tempo médio por acesso: | 1.03 ns |
| Array: 512   | KB | Tempo médio por acesso: | 1.21 ns |
| Array: 1024  | KB | Tempo médio por acesso: | 1.24 ns |
| Array: 2048  | KB | Tempo médio por acesso: | 1.24 ns |
| Array: 4096  | KB | Tempo médio por acesso: | 1.24 ns |
| Array: 8192  | KB | Tempo médio por acesso: | 1.35 ns |
| Array: 16384 | KB | Tempo médio por acesso: | 1.94 ns |

## ANÁLISE DOS RESULTADOS

Comparando os tempos de acesso das três máquinas, podemos observar que o PC3 tem o menor tempo de latência para tamanhos de vetor até 64 bytes, justamente porque a sua L1 é a de maior capacidade. A partir desses 64 bytes, ocorre uma inversão: o PC1 começa a ser mais rápido até o último array, pois o PC1 tem capacidade maior para L2 em diante. Dos três, o PC2 teve o pior desempenho.

## COMPARAÇÃO ENTRE MULTIPLICAÇÃO DE MATRIZES

### O QUE SÃO MATRIZES?

Elas são estruturas de dados organizadas em linhas e colunas, muito utilizadas para representar informações numéricas. Em memória, os elementos da matriz ficam armazenados sequencialmente. A multiplicação de matrizes convencionais consiste em multiplicar as linhas de A pelas colunas de B, acumulando o resultado na posição correspondente da matriz C. O primeiro item de cada linha é sempre  $A[0][0]$ ,  $A[1][0]$ ,  $A[2][0]$ ,  $A[3][0]$  e  $A[4][0]$ .

### COMPARAÇÃO:

Nesse método **convencional**, quando se trata de uma mesma linha, há melhor aproveitamento de localidade espacial, pois puxamos um bloco inteiro de elementos consecutivos da RAM para a cache ( $A[0][0]$ ,  $A[0][1]$ ,  $A[0][2]$ ,  $A[0][3]$ ,  $A[0][4]$ ). Por outro lado, o acesso às colunas de B é menos eficiente, já que os elementos acessados estão mais distantes entre si na memória, aumentando a ocorrência de cache misses. Assim, o processador precisa buscar dados com mais frequência em níveis inferiores da hierarquia de memória, aumentando o tempo de execução.

Já na multiplicação de matrizes **por bloco**, também chamada de “tiling”, dividimos uma matriz em submatrizes menores que caibam dentro da cache, geralmente L1 ou L2. Assim, temos blocos que são carregados inteiros na cache e, portanto, a taxa de cache hit aumenta significativamente para acessar os dados dessa submatriz. Isso se torna mais rápido, mais eficiente e menos custoso em saltos, porque acessamos dados de uma região pequena que são trazidos de uma vez.

## TESTE 3: MULTIPLICAÇÃO DE MATRIZES CONVENCIONAL X OTIMIZADA X POR BLOCOS

### Método

No arquivo *matrix\_mult.c*, temos a aplicação de três métodos de multiplicação. A primeira é a convencional já mencionada, em que as linhas de uma matriz A são multiplicadas pelas colunas de outra B e tudo é colocado na respectiva posição correta de uma matriz C, com quebra da localidade espacial. Dessa forma, nós pulamos de linha a linha (vemos cada coluna) para percorrer as células, mesmo que isso signifique ter que carregar os dados da memória principal várias vezes.

A segunda é uma versão da primeira um pouco mais otimizada para o processador e cache, em que há um acesso sequencial dos elementos que estão na cache, colados um no outro no mesmo bloco. Em vez de saltarmos linha a linha, saltamos de elemento para elemento, e isso reduz muito a taxa de *cache misses*. Ademais, usamos uma variável temporária antes para usar nos loops e não ter que buscar esse elemento na memória a cada iteração. Portanto, esse algoritmo não espera percorrer uma célula inteira de cada vez, ela foca em atualizar cada linha inteira mesmo que para isso tenha que fazer uma varredura horizontal e deixar as células incompletas.

A terceira função implementa a técnica de multiplicação por blocos (blocking). Inicialmente, toda a matriz resultado C é zerada. Em seguida, os laços ii, jj e kk percorrem as matrizes em blocos de tamanho  $bs \times bs$ , em vez de processar todas as linhas e colunas de uma só vez. Para cada bloco selecionado, os laços i, k e j realizam a multiplicação em si. A variável temporária tmpa, assim como no segundo algoritmo, armazena o valor  $A[i][k]$  para evitar acessos repetidos à memória no loop. Como os elementos de um mesmo bloco são reutilizados várias vezes antes de sua substituição, a cache é melhor aproveitada, reduzindo a quantidade de acessos às camadas mais lentas da memória e melhorando o desempenho para matrizes de grande dimensão.

## ANÁLISE COM PERF DE CADA MÁQUINA

### O QUE É PERF?

Perf é uma ferramenta que permite o monitoramento do desempenho do sistema fazendo uma “ponte” entre o hardware e o software. Portanto, o perf capta dois tipos de eventos:

**Eventos de hardware:** são obtidos diretamente da PMU (*Performance Monitoring Unit*) e geram informações como o uso de ciclos, quantidade de *cache misses*, IPC (instruções por ciclo), erros de predição de branch, entre outros.

**Eventos de software:** monitora chamadas de sistema, *page faults*, mudanças de contexto, entre outros.

O perf consegue identificar gargalos de memória com baixo overhead, ou seja, sem causar danos à performance do processador.

### Resultados

Tamanho da Matriz: 4096 x 4096

Tamanho do Bloco: 16

Rodadas: 1

PC1 (Thalisia), CPU: Ryzen 5 3600, S.O: Linux Mint

Executando Convencional: Concluído em 657.421002 segundos.

Executando Convencional Otimizada: Concluído em 50.135343 segundos.



Executando em Blocos: Concluído em 41.375310 segundos.

--- Resultados ---

A multiplicação em blocos foi 1.21x mais rápida que a versão convencional otimizada.

Em relação a matriz convencional sem otimizações, a versão em blocos foi 15.89x mais rápida.

PC2(Analiza), CPU: Intel Core I7-1185G7, S.O: Ubuntu 26.04 LTS

Executando Convencional:

Concluído em 700.635070 segundos.

Executando Convencional Otimizada:

Concluído em 53.701714 segundos.

Executando em Blocos:

Concluído em 50.748554 segundos.

--- Resultados ---

A multiplicação em blocos foi 1.06x mais rápida que a versão convencional otimizada.

Em relação a matriz convencional sem otimizações, a versão em blocos foi 13.81x mais rápida.

PC3 (Melissa), CPU: Intel Core I9-10850K, S.O: Windows

Executando Convencional: Concluído em 818.996194 segundos.

Executando Convencional Otimizada: Concluído em 61.864976 segundos.

Executando em Blocos: Concluído em 52.093854 segundos.

--- Resultados ---

A multiplicação em blocos foi 1.19x mais rápida que a versão convencional otimizada

Em relação a matriz convencional sem otimizações, a versão em blocos foi 15.72x mais rápida.

## COMPARAÇÃO POR ANÁLISE DOS RESULTADOS

A multiplicação de matrizes padrão não é adequada ao acesso pela cache, pois exige a multiplicação entre linhas e colunas e causa muitas *cache misses*, dado que a localidade espacial armazena apenas os elementos de uma mesma linha e não os de uma mesma coluna. Os dois outros algoritmos são melhores: para matrizes de tamanho mediano, é mais recomendável usar a multiplicação otimizada, que já dá conta da busca de elemento por elemento em vez de linha a linha, elementos estes que já estão todos juntos no mesmo bloco de localidade espacial da cache e, portanto, geram mais *cache hits* do que *cache misses*. Por fim, o algoritmo da multiplicação por blocos é ideal para matrizes grandes, mas, por ter mais iterações, é mais custoso em casos pequenos e médios.

## Utilização do comando perf no PC1:

comando perf report :

| overhead | função                          |
|----------|---------------------------------|
| 97,88%   | multiply_conventional           |
| 0,78%    | multiply_blocked                |
| 0,32%    | multiply_conventional_optimized |

Com essa tabela conseguimos perceber que a função com mais caches misses ( a de maior overhead ) é a de multiply\_conventional.

comando perf annotate:

|       |                                              |
|-------|----------------------------------------------|
|       | // Loop interno realizando o somatório       |
|       | for (k = 0; k < n; k++) {                    |
|       | C[i * n + j] += A[i * n + k] * B[k * n + j]; |
| 0,00  | 58: movsd (%rax),%xmm0                       |
| 0,09  | mulsd (%rdx),%xmm0                           |
|       | for (k = 0; k < n; k++) {                    |
| 88,70 | add \$0x8,%rax                               |
| 0,46  | add %rsi,%rdx                                |
|       | C[i * n + j] += A[i * n + k] * B[k * n + j]; |
| 0,01  | addsd %xmm0,%xmm1                            |
| 1,19  | movsd %xmm1, (%rcx)                          |
|       | for (k = 0; k < n; k++) {                    |
| 9,53  | cmp %rdi,%rax                                |
| 0,02  | jne 58                                       |
|       | for (j = 0; j < n; j++) {                    |

Com o comando perf annotate, é possível visualizar em qual linha do código em assembly houve mais cache misses: add \$0x8, %rax, na função multiply\_conventional.

comando perf stat:

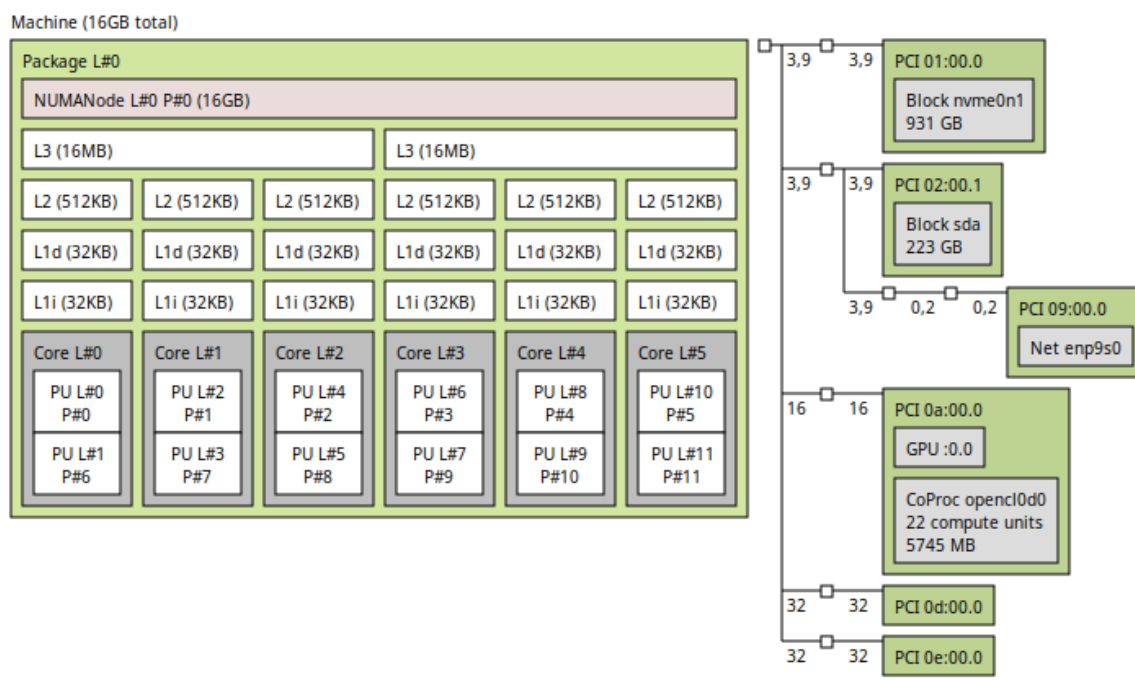
|                   |                      |                            |
|-------------------|----------------------|----------------------------|
| 3.101.529.145.589 | cycles               |                            |
| 1.702.270.516.670 | instructions         | # 0,55 ins per cycle       |
| 324.166.236.570   | cache-references     |                            |
| 84.924.869.811    | cache-misses         | # 26,20% of all cache refs |
| 749,119550155     | seconds time elapsed |                            |
| 747,927265000     | seconds user         |                            |
| 0,664819000       | seconds sys          |                            |

A execução da multiplicação convencional consumiu aproximadamente 3,10 trilhões de ciclos de CPU e executou 1,70 trilhão de instruções, resultando em uma média de 0,55 instruções por ciclo (IPC). Esse valor relativamente baixo indica que o processador passou uma parcela significativa do tempo aguardando dados chegarem da hierarquia de memória em vez de executar instruções efetivamente.

## Utilização do comando lstopo no PC1:

O comando **lstopo** exibe a topologia de hardware do sistema, incluindo núcleos, memória e diferentes níveis de cache. Essa ferramenta facilita a visualização da organização interna do computador e auxilia na análise da arquitetura e do desempenho da máquina. Os prints a seguir apresentam a topologia identificada no sistema utilizado.

### PC1 (Thalisia). CPU: Ryzen 5 3600. S.O: Linux Mint



Host: tali-B450M-S2H

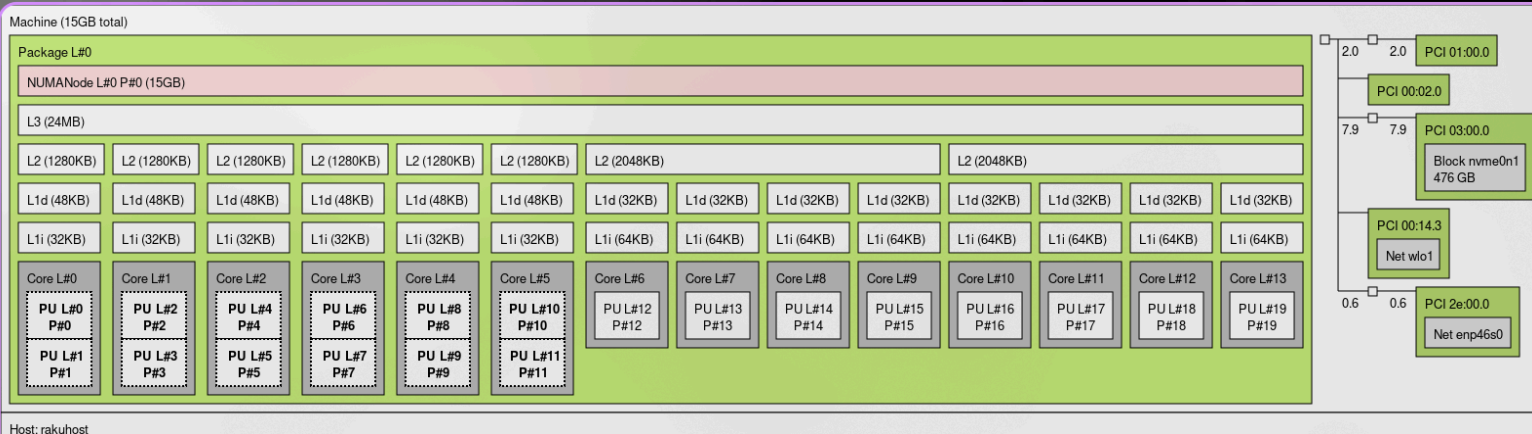
A seção referente ao PCI Express (PCIe) apresenta os dispositivos conectados aos barramentos de expansão da máquina. É possível identificar um SSD NVMe de 931 GB, um segundo dispositivo de armazenamento de 223 GB, a placa de rede (PCI 02:00.1) e a placa de vídeo (PCI 0a:00.0)

As PUs são as unidades de processamento dentro dos núcleos da CPU. Neste pc, temos 2 unidades de processamento por core (núcleo). O nome dessa tecnologia é Hyper-Threading/SMT.

Package L#0 significa que existe 1 processador físico (socket) instalado na máquina.

É possível visualizar que a L3 é compartilhada por 2 grupos com 3 cores, e que temos 16GB de RAM.

### PC4 (Haico). CPU: Intel core i7-12700H. S.O: Arch Linux



Host: rakuhost

Nesse computador, temos um sistema que dispõe de 20 unidades de processamento (PU). É possível identificar um SSD NVMe de 476 GB, uma interface de rede sem fio (wlo1), e uma interface de rede cabeada (enp46s0). 6 cores utilizam a tecnologia Hyper-Threading/SMT. É interessante notar também que a memória L3 de 24 MB é compartilhada por todos os 14 núcleos (cores) da CPU.

## REFERÊNCIAS BIBLIOGRÁFICAS

Repositório utilizado como referência para estudo e compreensão de diferentes implementações de multiplicação de matrizes, auxiliando na elaboração do Teste 3:

<https://github.com/cubiclesoft/matrix-multiply>

Artigo oferecido pelo professor no enunciado do trabalho, utilizado como referência teórica para o uso da ferramenta perf e análise de desempenho relacionado a cache misses e comportamento de memória em aplicações:

<https://developers.redhat.com/blog/2014/03/10/determining-whether-an-application-has-poor-cache-performance-2>

Demonstração interativa muito interessante que achamos na internet, utilizada para compreender o funcionamento de cache blocking e sua relação com a melhoria da localidade espacial:

<https://jukkasuomela.fi/cache-blocking-demo/>

Material utilizado para compreender métodos empíricos de medição do tamanho de linha de cache e sua relação com padrões de acesso à memória:

<https://lemire.me/blog/2023/12/12/measuring-the-size-of-the-cache-line-empirically/>

Fonte teórica utilizada para definição e entendimento do conceito de cache line size e seu papel na hierarquia de memória:

<https://www.sciencedirect.com/topics/computer-science/cache-line-size>